CodeQL unused-alert policy

This note is for developers who see CodeQL results in `third_party/**` and wonder whether they are engine bugs. The short answer: these files are not linked into the engine, and they are excluded for a reason.

What is excluded

The current CodeQL configuration excludes only generated copies and upstream tests, demos and tools:

Path	Why it is unused
`third_party/miniaudio-master/miniaudio-master/extras/miniaudio_split/**`	generated split copy of `miniaudio.h`; the engine compiles one header implementation in `Source/Engine/Audio/MiniAudioImpl.c`
`third_party/opus-master/opus-main/tests/**`	upstream Opus test programs
`third_party/opus-master/opus-main/celt/tests/**`	upstream Opus unit tests
`third_party/opus-master/opus-main/celt/opus_custom_demo.c`	Opus demo
`third_party/opus-master/opus-main/src/opus_compare.c`	Opus comparison tool
`third_party/opus-master/opus-main/dnn/adaconvtest.c`	Opus DNN test utility
`third_party/opus-master/opus-main/dnn/parse_lpcnet_weights.c`	Opus DNN weight parser tool
`third_party/opus-master/opus-main/dnn/dump_data.c`	Opus DNN dump utility
`third_party/flatbuffers-master/flatbuffers-master/tests/**`	upstream FlatBuffers tests

The exclusion lives in `.github/codeql/codeql-config.yml`. It is deliberately narrow: it does not hide the Opus codec, HarfBuzz, ImGui, ImPlot, Monocypher, miniaudio's real header or any other compiled vendor source.

Rules for developers

1. Do not add a new path to the exclusion list just to make the alert count go down. The path must be provably outside every build target.
2. If code starts including one of these files, remove its pattern first and triage the alerts again.
3. Do not replace miniaudio to silence the 29 split-copy alerts. The split copy

is generated from the same header; replacing the library is a separate backend project.

4. `stb_image_write` is no longer used: the engine's tools and tests use the in-tree `Fby::Assets::WritePng` image writer.
5. Used vendor alerts are patched or updated upstream; they are not excluded.

How to verify the exclusion

The configuration is read by the CodeQL workflow that scans `main`. Inspect the analysis and confirm that every remaining result is in a compiled vendor file:

```
.github/codeql/codeql-config.yml
Documentation/design/codeql-audit.md
```

The audit document records the latest analysis, the rule, and the split between compiled and unused vendor alerts.

When this policy should change

Revisit the list when a vendored dependency is updated, when a new backend is added, or when an excluded file becomes part of a target. Remove stale patterns in the same change that starts using the file.